Tips for ACA

- Use a language you are comfortable with
    - Coursework about writing a simulator, not learning a new language
    - OOP languages work well
    - Advise against things like Verilog
- You can think of your simulators as a state machine, a tick of the core is advancing the architectural state
    - Doesn't need to be architecturally 1-to-1, you can abstract certain features
        - E.g. the clock doesn't need a high and low signal
- Start simple and work your way up
    - A well-formed/developed base can help add newer features in the future
    - Think about what your simulator will be in a few weeks and develop for it
        - E.g. being able to add multiple execution units
    - Define a skeleton for your codebase and adapt it in-line with the coursework points/description
- Source control, commit little and often
    - By nature, simulators of this type can get confusing and complicated quickly, significant changes in emergent behaviour can stem from a small change in one pipeline unit
        - Reverting to early commits can be a good way to debug as a last resort
    - Great for trying stuff out and reverting when it doesn't work as intended
- Don't overcomplicate your instruction set
    - Stick with the basic set even if you're copying from a well-established ISA
    - You'll be using it to write your workloads by hand so justify to yourself why you'll need each instruction
- Experiments are just as important as your simulator marks wise (a ⅓ each)
    - Don't leave until the last minute
    - You may develop certain features in your simulator that target specific experiments so knowing what they'll be can help this
        - E.g. comparing branch predictor algorithms
- Having the functionality to output debug information for the state of your simulator helps resolve problems and ensures instruction flows are as expected
    - Examples would be outputting the register file, instructions in each pipeline unit, OoO queue entries
    - It could also have a stepping mode which steps through the simulation cycle-by-cycle
    - Don't spend all your time on this
- Recommend re-simulating your binaries after each new feature to ensure your simulator is functionally correct
- When going OoO, LoadStoreQueues can be useful for tracking in-flight memory operations and ensuring correct retirement ordering
    - It is worth looking into if not brought up in the lectures
- You don't need to write a compiler from (say) C to your ISA.
    - Writing programs in assembly and making a simple text parser is most commonly done

Lab format

- More like a set of drop-in/help/Q&A sessions than an actual lab with worksheets
- Come along to get help with your coursework
    - Both in terms of writing the simulator and understanding hardware features/concepts
- No set outline or assigned work is given in each lab
    - Solely determined by questions asked
- Won't necessarily be able to help with coding itself, mostly depends on whether you're using a language we understand
- The reason we started doing labs was to help curve the issues surrounding a very open-ended coursework and provide a soft structure for how to progress with it. Mainly intended to complement/build upon the written coursework assignment